

⚡ Fast CDP Testing Architecture & Reference Guide

Sub-2s Execution

Zero Sleep Hacks

Hot-Socket CDP

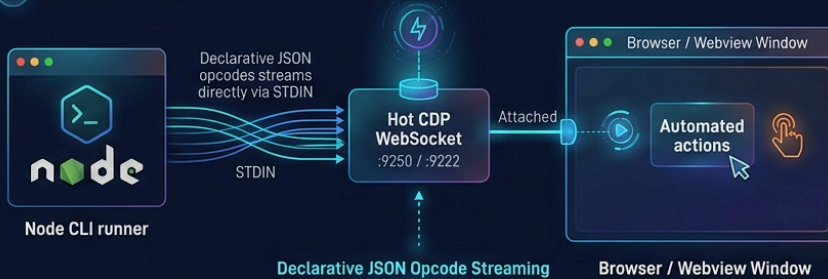
In-DOM Telemetry

Declarative JSON Opcodes

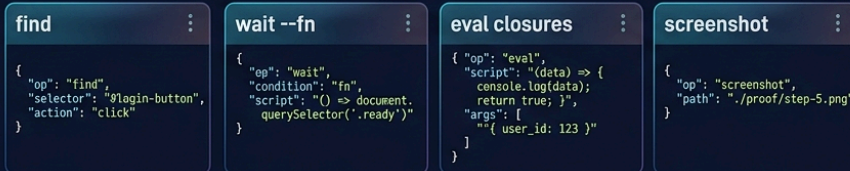
PREMIUM INFOGRAPHIC

Fast CDP Testing Architecture: Declarative JSON Opcode Streaming

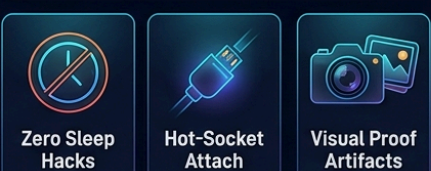
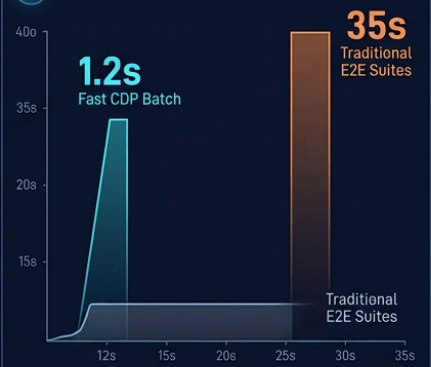
1 ARCHITECTURE DIAGRAM



2 SYNTAX CARDS (declarative JSON test opcodes)



3 SPEED COMPARISON



1. Core Architectural Pillars

1. Hot-Socket Connection (Port 9250 / 9222)

Attaches directly to an already-running desktop app or browser via WebSocket. Zero browser cold-starts (< 15ms attach vs 10s fresh launch).

2. JSON Opcode Stream over STDIN

All steps are passed as a single JSON array to `agent-browser batch --bail`. 40+ atomic steps run in a tight synchronous in-memory loop.

3. Reactive In-DOM Telemetry

Zero `sleep()` slop. Asserts DOM dataset contracts (`data-mesh-count="198"` , `data-hydration-status="ready"`) instantly on change.

4. In-Engine Layout Closures

A single in-browser JavaScript evaluation checks 50+ elements for touch target sizing ($\geq 44\text{px}$), clipping, and viewport overflow in 0.2ms.

2. Performance Benchmark Comparison

Attribute	Traditional E2E (Selenium / Cypress / Playwright Runner)	Fast CDP Batch Testing
Startup Overhead	5–15 seconds (cold browser boot & runner initialization)	< 15 milliseconds (hot-attached CDP WebSocket)
Full Suite Speed	30–60 seconds per scenario	1–2 seconds per scenario
Timing Strategy	Arbitrary <code>sleep()</code> hacks or hanging <code>networkidle</code>	Microtask reactive DOM predicates (<code>wait --fn</code>)
3D / Canvas Validation	Slow, brittle visual diffing or un-tested black box	Instant DOM dataset attribute telemetry
Layout & Bounds Checking	Dozens of round-trip IPC bridge queries	1 native V8 geometric closure (0.2ms)
Failure Handling	Cascading timeout retries (minutes of waiting)	Instant <code>--bail</code> termination on first broken invariant

3. Declarative Opcode Specification

Opcode	Syntax	Description
<code>tab</code>	<code>["tab", "t1"]</code>	Target specific browser/webview tab.
<code>find</code>	<code>["find", "role", "button", "click", "--name", "Save", "--exact"]</code>	Accessible semantic element click.
<code>fill</code>	<code>["fill", "#input-id", "Value"]</code>	Input text typing without simulated delays.
<code>press</code>	<code>["press", "Escape"]</code>	Dispatch native keyboard events.
<code>wait</code>	<code>["wait", "--fn", "<js-expression>"]</code>	Reactive wait until expression returns truthy.
<code>eval</code>	<code>["eval", "<js-code>"]</code>	Execute JS closure in browser V8; throws on error.
<code>set</code>	<code>["set", "viewport", "390", "844"]</code>	Dynamic viewport resize for responsive audits.
<code>screenshot</code>	<code>["screenshot", "path/to/image.png"]</code>	Direct GPU compositor framebuffer capture.
<code>reload</code>	<code>["reload"]</code>	Reload page to verify persistence/hydration.
<code>errors</code>	<code>["errors"]</code>	Assert zero uncaught browser console exceptions.

4. Universal Test Runner (`browser-batch.mjs`)

```
import { spawnSync } from 'node:child_process';
import { readFileSync, existsSync } from 'node:fs';
import { resolve } from 'node:path';

const [, , session, file, ...extra] = process.argv;
if (!session || !file) {
  console.error('Usage: node browser-batch.mjs <session-name> <scenario.json> [--cdp <port>]');
  process.exit(1);
}

const commands = JSON.parse(readFileSync(file, 'utf8'));
const cliCandidates = [
  process.env.AGENT_BROWSER_CLI,
```

```
    resolve('.temp/npm/_npx/8e62322f9a68a26a/node_modules/agent-browser/bin/agent-browser.js'),
    'agent-browser'
  ].filter(Boolean);

let cli = cliCandidates.find(c => existsSync(c)) || 'agent-browser';

const result = spawnSync(
  process.execPath,
  [cli, '--session', session, ...extra, 'batch', '--bail'],
  { input: JSON.stringify(commands), encoding: 'utf8', windowsHide: true, maxBuffer: 2e6, timeout: 180000,
    shell: process.platform === 'win32' }
);

process.stdout.write(result.stdout ?? '');
process.stderr.write(result.stderr ?? '');
if (result.error) console.error(result.error);
process.exit(result.status ?? 1);
```

5. Complete Production Scenarios

Scenario A: Standard Web App / SaaS Flow

```
[
  ["tab", "t1"],
  ["wait", "--fn", "!!document.querySelector('[data-hydration-status=ready]')"],
  ["fill", "input[name=email]", "dan@example.com"],
  ["fill", "input[name=password]", "SecurePass123!"],
  ["find", "role", "button", "click", "--name", "Sign In", "--exact"],
  ["wait", "--fn", "location.pathname === '/dashboard'"],
  ["eval", "if (!localStorage.getItem('auth_token')) throw Error('Missing auth token');"],
  ["screenshot", "screenshots/dashboard.png"],
  ["set", "viewport", "390", "844"],
  ["eval", "for (const b of document.querySelectorAll('button, a')) if (b.getBoundingClientRect().height < 44) throw Error('Touch target < 44px');"],
  ["screenshot", "screenshots/mobile-audit.png"],
  ["set", "viewport", "1600", "1000"],
  ["errors"]
]
```

Scenario B: 3D CAD & Native Desktop Flow (Tauri / WebView2)

```
[
  ["tab", "t1"],
  ["wait", "--fn", "!!document.querySelector('[data-hydration-status=ready]')"],
  ["eval", "if(!window.__TAURI_INTERNALS__) throw Error('Not native desktop app');"],
  ["find", "role", "button", "click", "--name", "Open 3D Model", "--exact"],
  ["wait", "--fn", "Number(document.querySelector('.canvas canvas')?.dataset.meshCount) === 198"],
  ["find", "role", "button", "click", "--name", "Front", "--exact"],
  ["find", "role", "button", "click", "--name", "Zoom in model", "--exact"],
  ["screenshot", "screenshots/native-front.png"],
  ["find", "role", "button", "click", "--name", "Fly", "--exact"],
  ["wait", "--fn", "document.querySelector('canvas')?.dataset.navigation === 'fly' &&
!!document.pointerLockElement"],
  ["press", "Escape"],
  ["wait", "--fn", "document.querySelector('canvas')?.dataset.navigation === 'orbit'"],
  ["reload"],
  ["wait", "--fn", "!!document.querySelector('[data-hydration-status=ready]')"],
  ["errors"]
]
```